

Why Split Your Database?

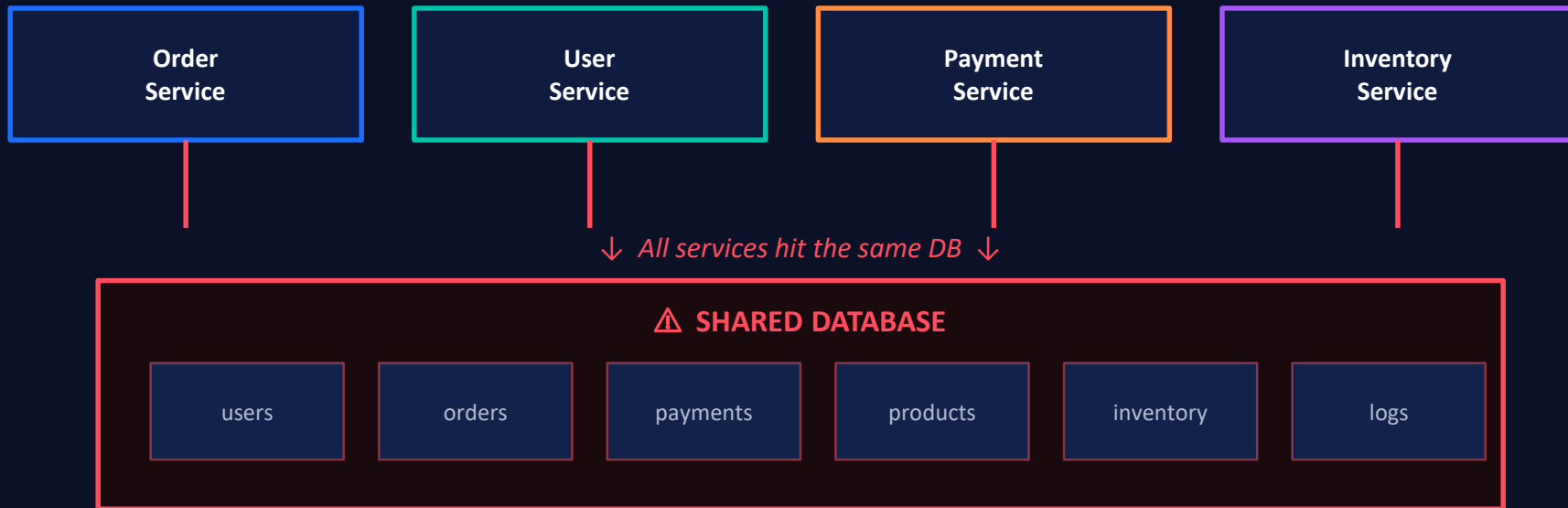
From one shared database to database-per-service

Schema Coupling

Scaling Problems

Data Ownership

The Shared Database Trap



Nobody owns anything. Everybody touches everything. One change can break the whole system.

Schema Change = Everyone's Problem

-- You rename a column in Orders

```
ALTER TABLE Orders  
RENAME COLUMN user_id TO customer_id
```



X Payment Service breaks

X Shipping Service breaks

X Reports Service breaks

X Analytics breaks

Coordination Tax

Every DB change needs sign-off from ALL teams before deployment

Fear of Change

Developers avoid refactoring because something unknown might break

Deployment Blocked

Team A can't ship until Team B finishes their pending migration

"You're afraid to delete ANYTHING because something, somewhere might depend on it."

PAIN POINT # 2

One Slow Query Kills Everyone

ProductService
runs heavy report

Locks Products
table 30 seconds

OrderService
tries to read Products

OrderService
hangs completely

 Users can't place orders — everything is down

PAIN POINT # 3

You Can't Scale Independently

 **Black Friday**

Orders traffic $\times 100$ — but you must scale the ENTIRE shared DB including Users, Products, Payments

 **Very Expensive**

Scaling a single massive DB is costly — you pay for resources used by services that don't need it

 **Tech Lock-in**

All services forced to use same DB tech — can't pick MongoDB for Users or Elasticsearch for Products

Database per Service

Golden Rule: Each service owns its data exclusively — no other service can touch it directly.



✓ No shared tables

✓ No cross-service joins

✓ Communicate only through APIs

"How do I get data from multiple services?"

Option A — API Composition

Client asks for order + user details

- 1 API Gateway receives request
- 2 Calls OrderService → gets order data
- 3 Calls UserService → gets user data
- 4 Merges & returns to client

✓ Simple • No messaging needed • Good for reads

Option B — Events (Messaging)

User updates email → all services sync

- 1 UserService updates email in its DB
- 2 Publishes event: UserEmailUpdated
- 3 RabbitMQ / Service Bus delivers event
- 4 OrderService listens & updates local copy

✓ Decoupled • Eventual consistency • RabbitMQ in .NET

Shared DB vs Database per Service

	✗ Shared Database	✓ DB per Service
Schema change	Breaks all services	Only affects one service
Slow query	All services hang	Isolated to one service
Deployment	Everyone waits	Teams ship independently
Scaling	Scale entire DB	Scale only what's needed
Data ownership	Nobody / everybody	Clear per service
Tech choice	One DB for all	Right tool per service